

**ISEP** INSTITUTO SUPERIOR  
DE ENGENHARIA DO PORTO

# JavaScript



Sistemas de Informação 1

# Content

## 1. Context

## 2. Syntax

## 3. Data types

1. Primitive
2. Non primitive
3. Type Conversion

## 4. Operators

1. Arithmetic
2. Assignment
3. Comparison
4. Logical

## 5. Structures

1. Conditional
2. Cycles

## 6. Functions

1. Syntax
2. Return
3. Variable scope
4. User interface

## 7. Arrays

1. Syntax
2. Adding elements
3. Removing elements
4. Iterating
5. Ordering
6. Search methods
7. Transformation methods

## 8. Objects:

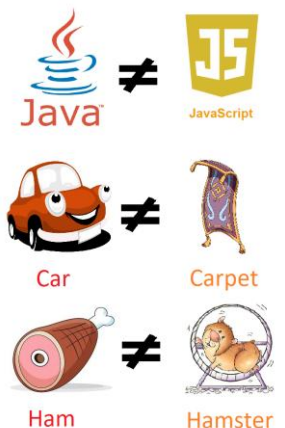
1. Creation and properties
2. Accessing and changing properties
3. Methods
4. Object copy

## 9. Classes

1. Syntax
2. Example

# Context

- **Java** ≠ **JavaScript**
- **JavaScript** created in 1995 by Brendan Eich for the Netscape Navigator 2
- Its first name was LiveScript but because of Java's popularity at the time, the name was changed
- In 1997 it became a European Computer Manufacturers Association (ECMA) standard
  - ECMAScript: <https://www.ecma-international.org/publications/standards/Ecma-262.htm>
- **JavaScript** é is a scripting language that follows the **ECMAScript** standard
- It became popular as a programming language used for browsers, but it is now also used for other purposes (e.g: Node.js)
- To see JavaScript console in any browser: **Ctrl + Shift + I**



# Syntax

```
// comment
```

```
/* another  
comment */
```

```
let username = "Maria" // the value in variable username is "Maria"
```

```
{  
let x = 1 // x is initialised with 1  
x = 2 // x now has the value 2  
let x = 3 // error: x identifier was already used  
}
```

Variables declared with `let` have local scope (in the block)

**Variable names**

- May contain characters, numbers, symbols (\$, \_).
- The 1<sup>st</sup> character can not be a number
- Are *case-sensitive*
- Can not be reserved words (JavaScript keywords)

```
let x = 1 // x is initialised with 1  
{  
let x = 2 // no error. x now has the value 2  
}  
// x has the value 1 again
```

The variable `x` only has the value 2 inside the block

```
const yearOfBirth = 2000  
yearOfBirth = 2001 // error: constant values can not be changed
```

# Primitive data types

JavaScript is a dynamically typed language: a variable may contain data in any type. Example:

```
let message = "hello"
message = 123 // changing the data type is valid
```

Primitive data types:

`typeof x`

`typeof(x)`

- **number**: integer and decimal numbers

- Operations: +, -, \*, /

```
"2" * 3 // 6
```

- Special values: Infinity (1/0), -Infinity (-1/0), NaN ("hello"\*3)

```
isNaN("hello"*3) // true
```

- **string**: sets of characters (words, text)

- Backticks

```
const name = "Maria"
`Hello ${name}!` //Result: Hello Maria!
```

- Concatenating (+)

```
"1" + 2 // 12
```

- **boolean**: true / false
- **null** (unknown)
- **undefined** (not yet defined)

# Non primitive data types

- **object**: more complex data structures

```
let person = {  
  firstName: "Maria",  
  lastName: "Sousa",  
  age: 37,  
  favouriteColor: "purple"  
}
```

- **symbol**: exclusive identifiers
  - May be used to identify objects' properties

# Type conversion

- To String

```
let age = String(12)    // "12"
```

- To number

```
let age = Number(12)    // 12
Number("  123  ")      // 123
Number("casa")          // NaN
Number("")              // 0
Number("true")          // 1
Number("false")         // 0
```

- To Boolean

```
let eyes
Boolean(eyes)           // false
eyes = 2
Boolean(eyes)           // true
```

# Operators

In JavaScript there are several different operator types:

- **Arithmetic:** used for arithmetic operations
- **Assignment:** used to assign values to the variables
- **Comparison:** used to determine equalities between variables or values
- **Logical:** used to determine logical properties between variables or values



# Arithmetic operators

| Operator                | Description                                | Example                           |
|-------------------------|--------------------------------------------|-----------------------------------|
| <code>+, -, *, /</code> | Sum, subtraction, multiplication, division | <code>3+2 // 5</code>             |
| <code>%</code>          | Module                                     | <code>12%10 // 2</code>           |
| <code>**</code>         | Power                                      | <code>3**2 // 9</code>            |
| <code>++, --</code>     | Increment, decrement                       | <code>let y=3; y++; y // 4</code> |

# Assignment operators

| Operator                        | Description                | Example                              |
|---------------------------------|----------------------------|--------------------------------------|
| =                               | Simple assignment          | <code>let x = 3 // 3</code>          |
| <code>+=, -=, *=, /=, %=</code> | Assignment with operations | <code>let x = 3; x+=2; x // 5</code> |

# Comparison operators

| Operator    | Description                                              | Example                                                                                 |
|-------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------|
| ==          | Equals (value)                                           | <pre>2==2 // true 2==3 // false</pre>                                                   |
| ===         | Equals (value and type)                                  | <pre>2===2 // true 2===`2` // false</pre>                                               |
| !=          | Different (value)                                        | <pre>2!=2 // false 2!=3 // true</pre>                                                   |
| !==         | Different (value or type)                                | <pre>2!==2 // false 2!==`2` // true 2!==3 // true</pre>                                 |
| >, >= <, <= | Greater than, greater or equal, less than, less or equal | <pre>2&gt;2 // false 2&gt;=2 // true 2&gt;1 // true `Porta` &gt; `Porto` // false</pre> |
| ?           | Ternary operator                                         | <pre>Math.PI &gt; 4 ? "S" : "N"; // "N"</pre>                                           |

# Logical operators

| Operator | Description | Example                                                                                             |
|----------|-------------|-----------------------------------------------------------------------------------------------------|
| & &      | Logical AND | false && false // false<br>false && true // false<br>true && false // false<br>true && true // true |
|          | Logical OR  | false    false // false<br>false    true // true<br>true    false // true<br>true    true // true   |
| !        | Logical NOT | !true // false<br>!false // true                                                                    |

# Conditional structures

## if

```
if(condition){
    instructions
}
```

```
if(hour <= 12){
    greeting = "Bom dia"
}
```

## if / else if / else

```
if(condition){
    instructions-if
} else if (condição2) {
    instruções-if2
} else {
    instructions-else
}
```

```
if(hour <= 12){
    greeting = "Bom dia"
} else if(hour<=20){
    greeting = "Boa tarde"
} else {
    greeting = "Boa noite"
}
```

## if / else

```
if(condition){
    instructions-if
} else {
    instructions-else
}
```

```
if(hour <= 12){
    greeting = "Bom dia"
} else {
    greeting = "Boa tarde"
}
```

## switch

```
switch(value){
    case ...: instructions; break;
    ...
}
```

```
switch(new Date().getDay()){
    case 0: day = "Domingo"; break;
    case 1: day = "Segunda"; break;
    case 2: day = "Terça"; break;
    case 3: day = "Quarta"; break;
    case 4: day = "Quinta"; break;
    case 5: day = "Sexta"; break;
    case 6: day = "Sábado"; break;
}
```

# Cycle structures

## for

```
for(initialization; condition; update) {  
    instructions  
}
```

```
for(let i=0; i<3; i++){  
    alert(i)  
}
```

## do-while

```
do {  
    instructions  
} while(condition)
```

```
let i=0  
do {  
    alert(i)  
    i++  
} while(i<3)
```

## while

```
while(condition) {  
    instructions  
}
```

```
let i=0  
while(i<3){  
    alert(i)  
    i++  
}
```

# Functions

- **JavaScript function:** block of code designed to perform a specific task and can be reused

- The code becomes smaller, more readable and easier to maintain

- **Syntax:**

```
function name (param1, param2, ...) {  
    code to execute  
}
```

- **Example:** declaring a function:

```
function showMessage () {  
    let message = "Olá!"  
    console.log(message)  
}
```

- **Example:** calling a function:

```
showMessage() // Result: Olá!
```

# Return

```
function sum(a,b) {  
    return a + b  
}
```

```
let result1 = sum(4,3)  
result1      // 7  
let result2 = sum(4)  
result2      // NaN
```

```
function sum2(a, b = 0) {  
    return a + b  
}
```

```
let result3 = sum2(4)  
result3     // 4
```



# Variable scope

## ■ Global

```
let nome = "Ana"
function mostraMensagem() {
  nome = "Maria"
  let mensagem = `Olá ${nome}`
  console.log(mensagem)
}
```

```
console.log(nome) // Ana
mostraMensagem() // Olá Maria
console.log(nome) // Maria
```

## ■ Local

```
let nome = "Ana"
function mostraMensagem() {
  let nome = "Maria"
  let mensagem = `Olá ${nome}`
  console.log(mensagem)
}
```

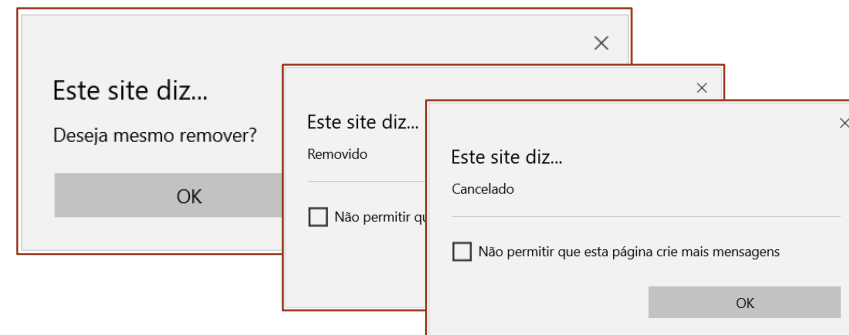
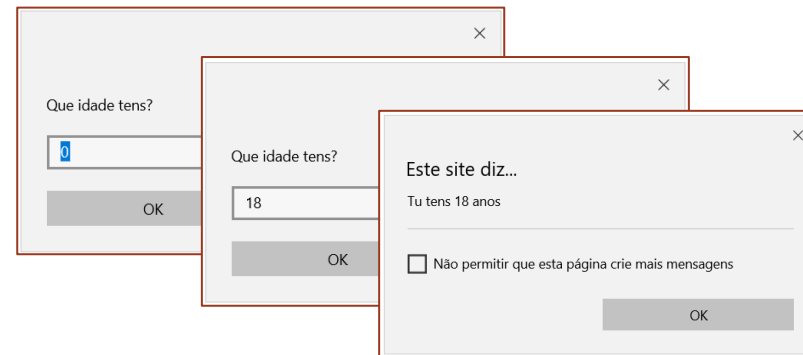
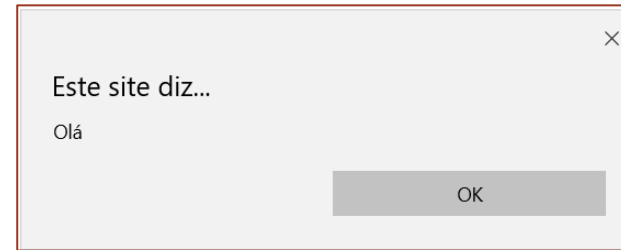
```
console.log(nome) // Ana
mostraMensagem() // Olá Maria
console.log(nome) // Ana
```

# User interface functions

```
alert("Hello")
```

```
let idade = prompt("How old are you?",0)
if(idade!=null) {
    alert(`You are ${idade} years old`)
}
```

```
let removerRegisto = confirm("Remove?")
if(removerRegisto) {
    alert("Removed")
} else {
    alert("Cancelled")
}
```



# Arrays

- Declaration

```
let countries = new Array()
```

```
let countries = []
```

```
let countries = ["Portugal", "Espanha", "França"]
```

|          |         |        |
|----------|---------|--------|
| Portugal | Espanha | França |
| 0        | 1       | 2      |

- Operations

```
console.log(countries.length) // 3
```

```
console.log(countries[1]) // Espanha
```

```
console.log(countries()) // [object Array]: ["Portugal", "Espanha", "França"]
```

```
let outro = countries // copy the arrays by reference
```

```
console.log(outro === countries) // true
```

```
outro[3] = "Alemanha" // modifies the array
```

```
console.log(countries) // [object
```

```
Array]: ["Portugal", "Espanha", "França", "Alemanha"]
```

|          |         |        |          |
|----------|---------|--------|----------|
| Portugal | Espanha | França | Alemanha |
|----------|---------|--------|----------|

# Array manipulation: adding elements

## Example: add elements

`countries[3] = "Alemanha"`

|          |         |        |          |
|----------|---------|--------|----------|
| Portugal | Espanha | França | Alemanha |
| 0        | 1       | 2      | 3        |

`countries[5] = "Itália"`

|          |         |        |          |           |        |
|----------|---------|--------|----------|-----------|--------|
| Portugal | Espanha | França | Alemanha | undefined | Itália |
| 0        | 1       | 2      | 3        | 4         | 5      |

## Useful operation

- **Push:** adds elements to the end of the array. Returns its new size.

`countries.push("Alemanha") // 4`

|          |         |        |          |
|----------|---------|--------|----------|
| Portugal | Espanha | França | Alemanha |
| 0        | 1       | 2      | 3        |

- **Unshift:** adds elements to the end of the array. The other elements are shifted forward. Returns its new size.

`countries.unshift("Itália") // 5`

|        |          |         |        |          |
|--------|----------|---------|--------|----------|
| Itália | Portugal | Espanha | França | Alemanha |
| 0      | 1        | 2       | 3      | 4        |

- **Splice:** adds/removes elements without leaving empty spaces

- **syntax:** `arrayName.splice(index, 0, elementToAdd1, elementToAdd2, ...)`

`countries.splice(1,0,"Polónia","Marr`

|        |         |          |          |         |        |          |
|--------|---------|----------|----------|---------|--------|----------|
| Itália | Polónia | Marrocos | Portugal | Espanha | França | Alemanha |
| 0      | 1       | 2        | 3        | 4       | 5      | 6        |

**Note:** operations at the end of the array are better, because they are faster

# Array manipulation: removing elements

## Example: remove elements

```
delete countries[1]
```

|          |           |        |
|----------|-----------|--------|
| Portugal | undefined | França |
| 0        | 1         | 2      |

|          |         |        |
|----------|---------|--------|
| Portugal | Espanha | França |
| 0        | 1       | 2      |

## Useful operations

- **Pop:** removes the array's last element. Returns the element.

```
countries.pop() // Alemanha
```

|        |         |          |         |        |          |
|--------|---------|----------|---------|--------|----------|
| Itália | Polónia | Marrocos | Espanha | França | Alemanha |
| 0      | 1       | 2        | 3       | 4      | 5        |

- **Shift:** removes the array's first element. Other elements shift back. Returns the element.

```
countries.shift() // Itália
```

|         |          |         |        |
|---------|----------|---------|--------|
| Polónia | Marrocos | Espanha | França |
| 0       | 1        | 2       | 3      |

- **Splice:** adds/removes elements without leaving empty spaces

- **syntax:** `arrayName.splice(index, nrOfElementsToRemove)`

```
countries.splice(1,1) // Marrocos
```

|         |         |        |
|---------|---------|--------|
| Polónia | Espanha | França |
| 0       | 1       | 2      |

```
countries.splice(0,2) // Polónia, Espanha
```

|        |
|--------|
| França |
| 0      |

**Note:** operations at the end of the array are better, because they are faster

## Other methods to add/remove array elements

```
let paises = ["Portugal", "Espanha", "França", "Itália", "Alemanha"]
```

| Method                           | Description                                                                           | Example                                                                                                  |
|----------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>slice(start, end)</code>   | Creates a new array with the elements between the <b>start</b> and <b>end</b> indexes | <pre>let menosPaises = paises.slice(1,3) console.log(menosPaises)           // "Espanha", "França"</pre> |
| <code>concat(...items...)</code> | Joins items to the array                                                              | <pre>menosPaises.concat("Marrocos") // "Espanha", "França", "Marrocos"</pre>                             |

# Iterating

```
let paises = ["Portugal", "Espanha", "França"]
```

```
let size = paises.length
for(let i=0; i<size; i++) {
  console.log(paises[i])
}
```

```
for(let index in paises) {
  console.log(paises[index])
}
```

```
for(const pais of paises) {
  console.log(pais)
}
```

```
paises.forEach((item, index, array) => {
  alert(`${item} is at index ${index} in ${array}`)
});
```

# Sorting

```
let countries = ["Portugal","Espanha","França"]  
countries.sort() // "Espanha", "França", "Portugal" (alphabetical order)  
countries.reverse() // "Portugal", "França", "Espanha"
```

```
let numbers = [12,7,10,18,5]  
numbers.sort() // 10, 12, 18, 5, 7 (alphabetical order)
```

```
function ascOrder(a, b) {  
  return a - b  
}
```

Gives a different order

```
numbers = [12,7,10,18,5]  
numbers.sort(ascOrder) // 5, 7, 10, 12, 18 (ascending order)
```

```
numbers.sort(function(a,b){return a - b}) // 5, 7, 10, 12, 18 (same, but smaller)
```



# Search methods

```
let paises = ["Portugal", "Espanha", "França", "Itália", "Alemanha"]
let numeros = [1, 2, 3, 4, 5, 4, 3, 2, 1, 2, 3, 4, 5]
let idades = [12, 20, 40, 15, 17, 31]
```

```
function isAdult(idade) {
  return idade >= 18
}
```

| Method                             | Description                                                                                           | Example                                                                                                                         |
|------------------------------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>indexOf(element, pos)</code> | Searches for the <b>element</b> on the array starting on index <b>pos</b> and returns the index or -1 | <pre>paises.indexOf("Espanha") // 1 paises.indexOf("Inglaterra") // -1 numeros.indexOf(2) // 1 numeros.indexOf(2, 3) // 7</pre> |
| <code>lastIndexOf(element)</code>  | Searches for the <b>element</b> on the array and returns the index of the last occurrence or -1       | <pre>numeros.lastIndexOf(2) // 9 numeros.lastIndexOf(7) // -1</pre>                                                             |
| <code>includes(element)</code>     | Determines if <b>element</b> belongs to the array                                                     | <pre>numeros.includes(3) // true numeros.includes(7) // false</pre>                                                             |
| <code>find(function)</code>        | Searches elements through <b>function</b> and returns the 1 <sup>st</sup> found                       | <pre>idades.find(isAdult) // 20</pre>                                                                                           |
| <code>filter(function)</code>      | Searches elements through <b>function</b> and returns all found                                       | <pre>idades.filter(isAdult) // [20, 40, 31]</pre>                                                                               |

# Array transformation methods

```
let numeros = [1,2,3,4,5]
```

```
function mult2(nr) {  
  return nr*2  
}
```

```
function soma (acumulado, atual) {  
  return acumulado + atual  
}
```

| Method                               | Description                                                                                       | Example                                                                                                                  |
|--------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>map(function)</code>           | Applies <b>function</b> to all the elements in the array and returns the result                   | <code>numeros.map(mult2)</code> // <code>[2,4,6,8,10]</code>                                                             |
| <code>fill(value, start, end)</code> | Fills the arrays positions from <b>start</b> to <b>end</b> with <b>value</b>                      | <code>numeros.fill(9,1,3)</code> // <code>[1,9,9,4,5]</code><br><code>numeros.fill(9)</code> // <code>[9,9,9,9,9]</code> |
| <code>join(separator)</code>         | Joins the array's elements separated by <b>separator</b> , into a string                          | <code>numeros.join("*")</code> // <code>1*2*3*4*5</code>                                                                 |
| <code>reduce(func, initValue)</code> | Applies <b>func</b> <sup>(1)</sup> from an <b>initValue</b> to reduce the array to a single value | <code>numeros.reduce(soma)</code> // <code>15</code>                                                                     |

<sup>(1)</sup>Function func must be like:

```
function(acumulator, current, index, array){...}
```

# Objects: creation and properties

## Object creation

```
let user1 = {}
```

```
let user2 = new Object()
```

} Empty objects

```
let user3 = {
```

```
  name: "Maria",
```

```
  age: 39
```

```
}
```

Properties

pairs key: value

```
let name = "Ana"
```

```
let age = 41
```

```
let user4 = {name, age}
```

## Property creation

```
let newProp = "gender"
```

```
let user5 = {
```

```
  [newProp]: "male"
```

```
}
```

## Property deletion

```
delete user3.age
```

```
console.log(user3)
```

```
// {name: "Maria"}
```

# Objects: access and modify properties

## Access / modify object's properties

```
console.log(user3.name) // "Maria"
```

```
user3.name = "Maria Matos"
```

```
console.log(user3) // {age: 39, name: "Maria Matos"}
```

```
let key = prompt("What do you want to know about the user?", "name")
alert(user3[key])
```

## Determine if a property exists

```
console.log("age" in user4) // true
```

```
console.log("eyeColor" in user4) // false
```

## Iterate through all object's keys

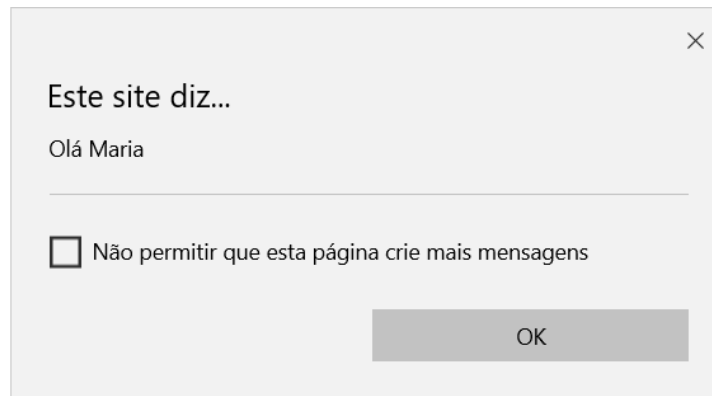
```
for(let key in user4){
  alert(key + ": " + user4[key])
}
```



# Objects: methods

```
let user = {  
  name: "Maria",  
  age: 39,  
  sayHello() {  
    alert(`Hello ${this.name}`)  
  }  
}
```

```
user.sayHello()
```



# Object copy

```
let person = {name = "Ana"}
```

- The copy is made by reference

```
let newPerson = person
newPerson.name = "Maria"
console.log(newPerson.name) // "Maria"
console.log(person.name)    // "Maria" (the value is modified in both objects)
```

- How to not copy by reference

```
let newPerson = {}
// copy all properties
for(let key in person) {
    newPerson[key] = person[key]
}
newPerson.name = "Maria"
console.log(newPerson.name) // "Maria"
console.log(person.name)   // "Ana" (unchanged)
```

- Another way

```
let newPerson = Object.assign({}, person)
newPerson.name = "Maria"
console.log(newPerson.name) // "Maria"
console.log(person.name)   // "Ana" (unchanged)
```

# Classes

- Used when several objects are needed

- Syntax

```
class MyClass {  
    constructor (...) {...}  
    method1 (...) {...}  
    method2 (...) {...}  
    get property (...) {...}  
    set property (...) {...}  
    static staticMethod (...) {...}  
  
    ...  
}
```

1<sup>st</sup> character is CAPS

- Create a class object

```
let myObject = new MyClass (...)
```

# Classes

- Example

```
class Person{
  constructor(name, age) {
    this.name = name
    this.age = age
  }
  get name(){
    return this._name
  }
  set name(value){
    this._name = value
  }
  sayHello() {
    alert(`Olá ${this._name}`)
  }
  static compare(personA, personB) {
    return personA.age - personB.age
  }
}
```

`_` : internal  
variable

```
let person1 = new Person("Maria", 39)
person1.sayHello()
```





**ISEP**

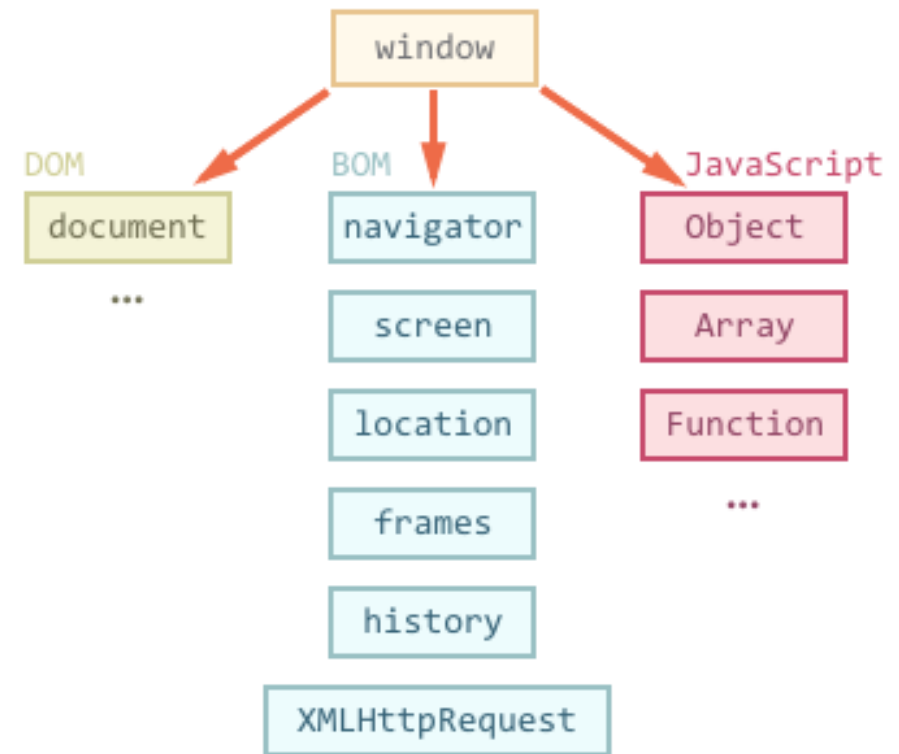
## **DOM: Document Object Model**

# Content

1. Context
  2. Integration of JavaScript into HTML
  3. Structure
  4. DOM methods examples
    1. getElementById(id)
    2. getElementsByTagName(name)
    3. getElementsByClassName(name)
    4. querySelector(selector)
    5. querySelectorAll(selector)
    6. createElement(element)
    7. createTextNode(text)
    8. appendChild(element)
    9. replaceChild(new, old)
    10. removeChild(element)
    11. Methods to change HTML elements
  5. Event types
  6. Examples
    1. onclick
    2. Verify matching passwords
-

# Context

- **DOM:** Document Object Model
- W3C standard
- Defines a uniform mean to access documents
- Divided in three parts
  - Core
  - XML
  - HTML
- JavaScript can access and alter all the elements of the HTML document and the Browser Object Model (BOM)



# Integrating JavaScript into HTML

## ■ Local



```
<html>
  <head>
    ...
  </head>
  <body>
    <script>
      alert("Ola")
    </script>
  </body>
</html>
```

## ■ External



```
<html>
  <head>
    ...
  </head>
  <body>
    ...
    <script src="path_to_file.js"></script>
  </body>
</html>
```

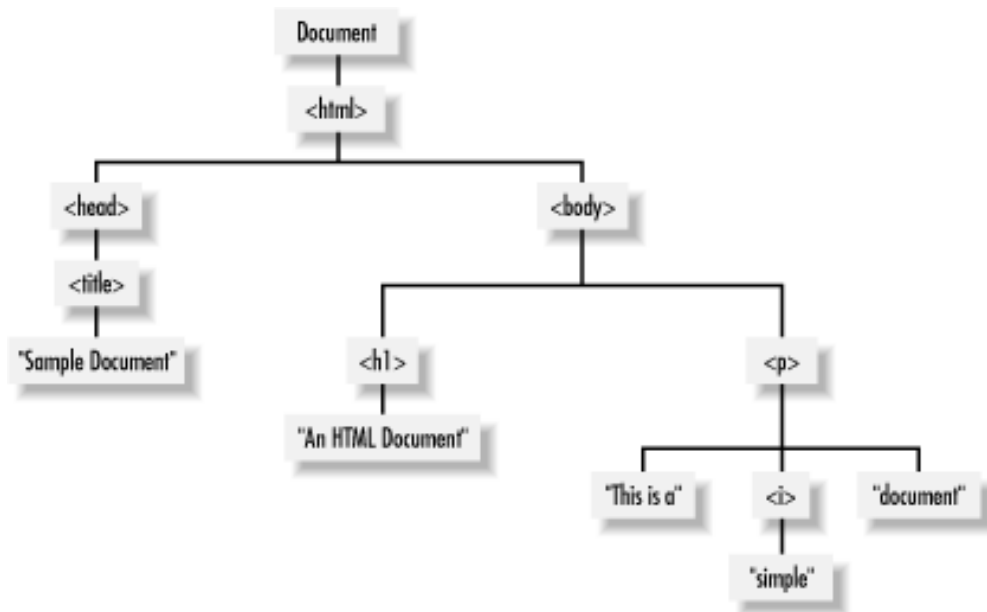
### Advantages:

- HTML / JavaScript separation
- Readability and maintenance of HTML and JavaScript code
- JavaScript files are cached → faster loading

# Structure

## ■ Tree of different types of objects:

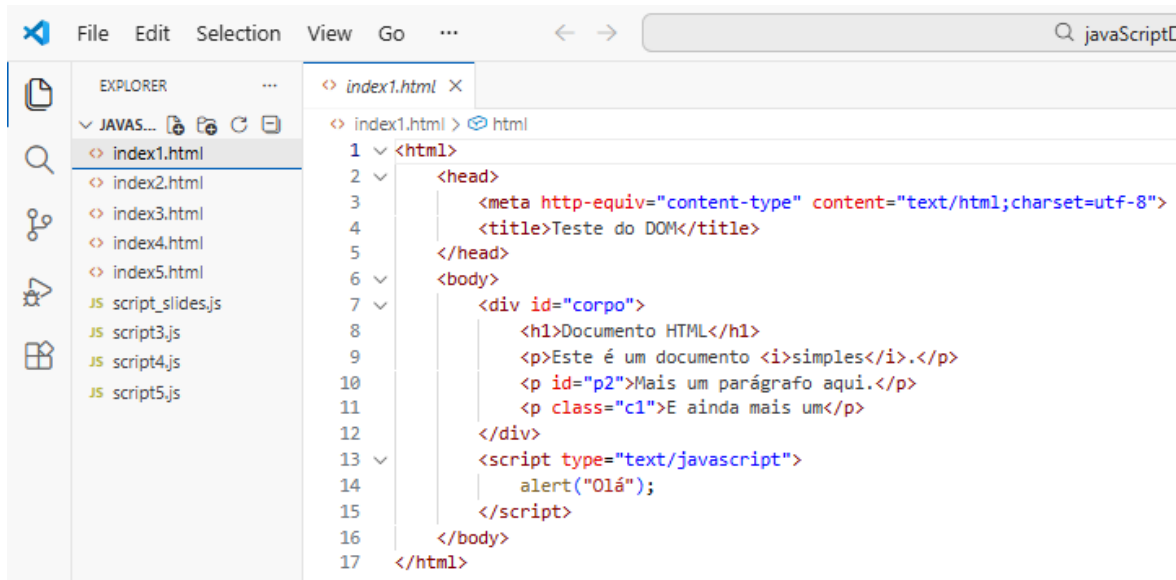
- **Document:** the document (fig: Document)
- **Node:** an HTML's generic node (fig: <title>, "Sample Document", <p>, ...)
- **NodeList:** the list of the nodes
- **Element:** elements created by HTML tags (fig: <html>, <head>, <title>, <body>, <h1>, ...)
- **Text:** text inside elements (fig: "Sample Document", "An HTML Document", "This is a", ...)



## ■ The tree is composed of **nodes**

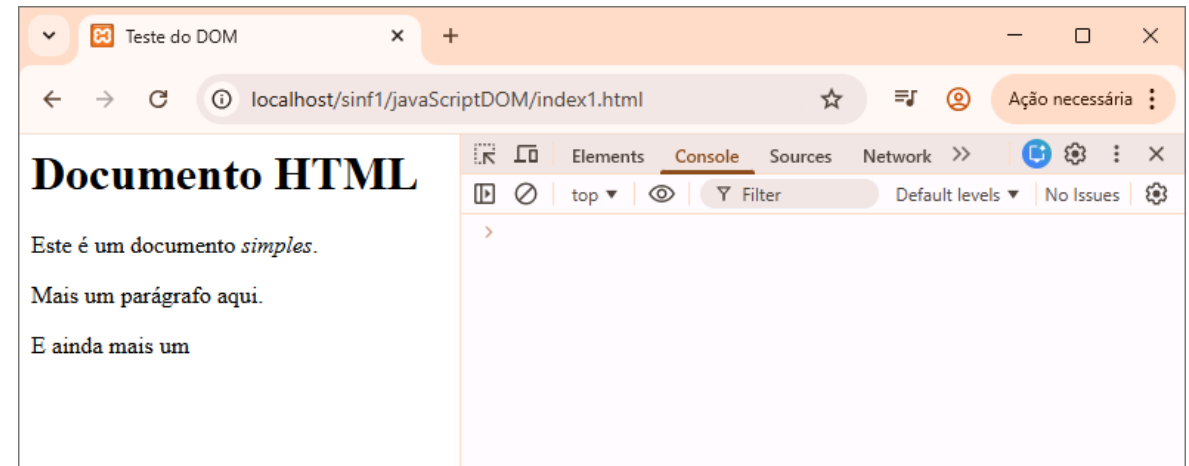
- The root is the Document node (no parent)
- All the other nodes have a **parent** node
- Document is parent of <html>
- <h1> is parent of "An HTML Document"
- Nodes can have **children**
- "Sample Document" is children of <title>
- "This is a" is children of <p>

# DOM examples: Starting point



The image shows a Visual Studio Code editor with the file explorer on the left displaying a project structure with files like index1.html through index5.html and several JavaScript files. The main editor area shows the source code of index1.html, which is an HTML document with a head section containing a meta tag and a title, and a body section containing a div with an id of 'corpo' and a script tag that alerts 'Olá'.

```
1 <html>
2 <head>
3   <meta http-equiv="content-type" content="text/html; charset=utf-8">
4   <title>Teste do DOM</title>
5 </head>
6 <body>
7   <div id="corpo">
8     <h1>Documento HTML</h1>
9     <p>Este é um documento <i>simples</i>.</p>
10    <p id="p2">Mais um parágrafo aqui.</p>
11    <p class="c1">E ainda mais um</p>
12  </div>
13  <script type="text/javascript">
14    alert("Olá");
15  </script>
16 </body>
17 </html>
```



# Accessing elements: getElementById(id)

## getElementById(id)

Finds the reference to an HTML element from its id attribute

```
document.getElementById("p2")
```

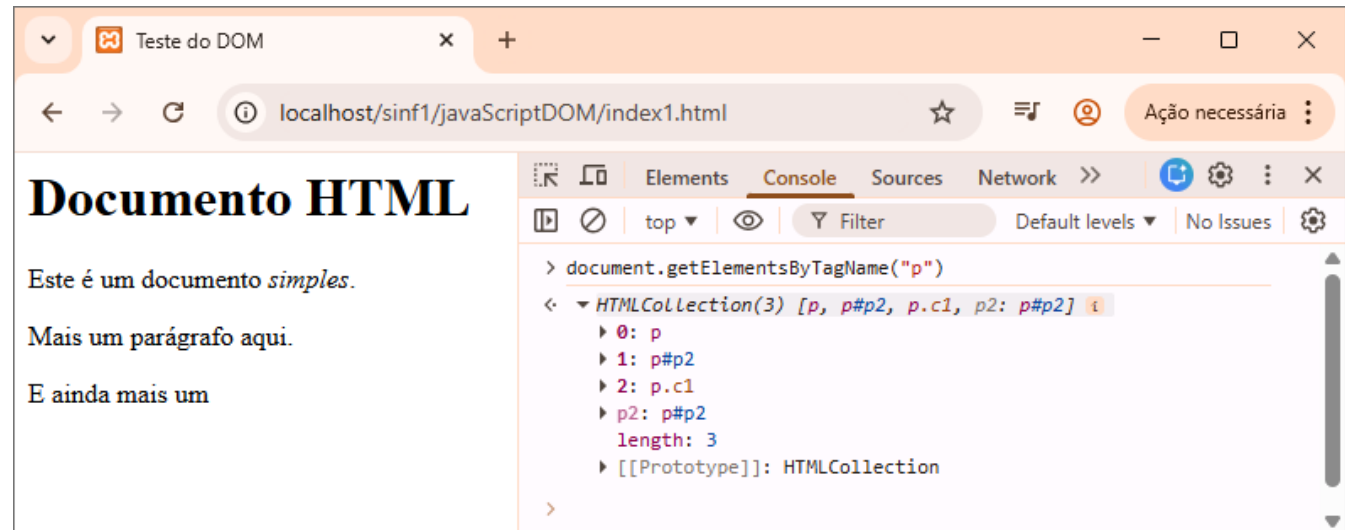


# Accessing elements: `getElementsByTagName(name)`

## `getElementsByTagName(name)`

Finds the reference to a set of HTML elements from their tags' name

```
document.getElementsByTagName("p")
```



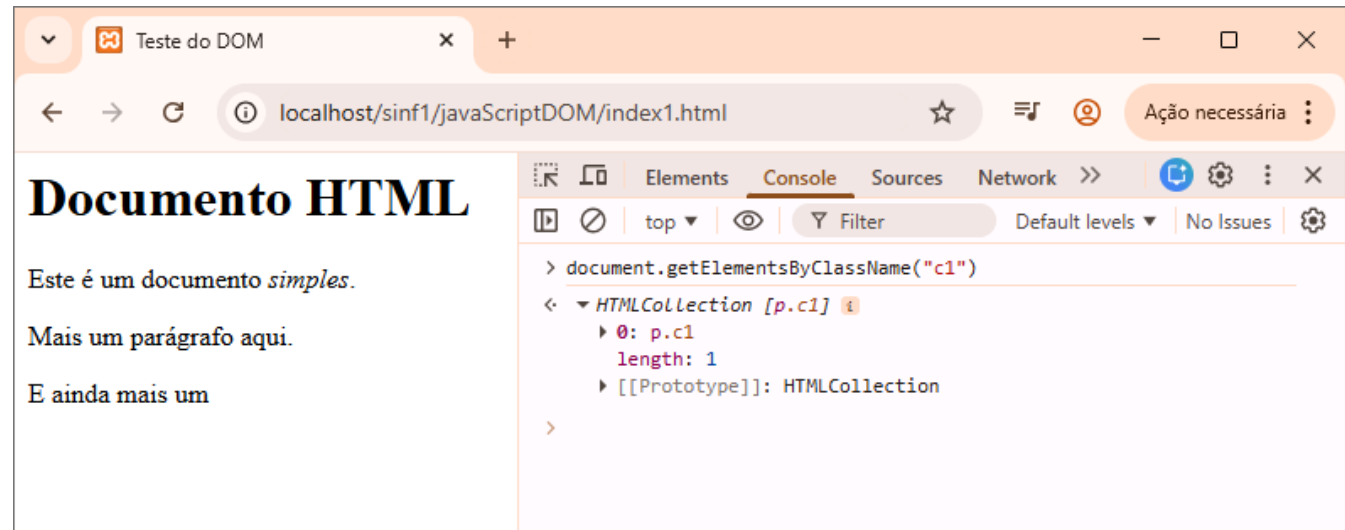


# Accessing elements: `getElementsByClassName(name)`

## `getElementsByClassName(nome)`

Finds the reference to a set of HTML elements from their class attribute

```
document.getElementsByClassName("c1")
```

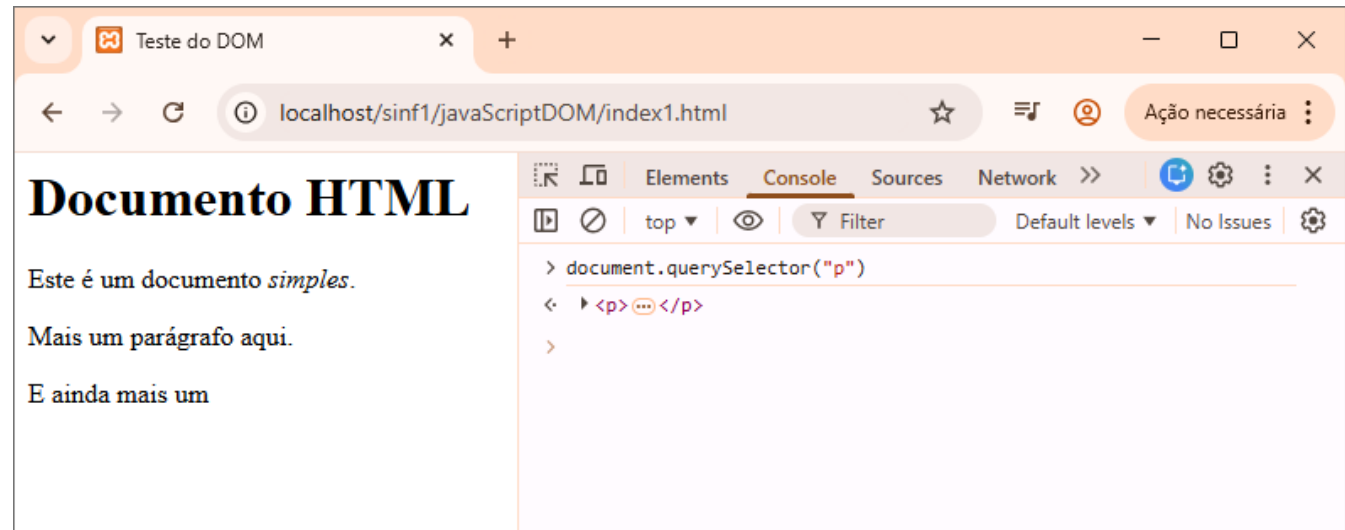


# Accessing elements: `querySelector(selector)`

## `querySelector(selector)`

Finds the reference to an HTML element from a CSS selector

```
document.querySelector("p")
```

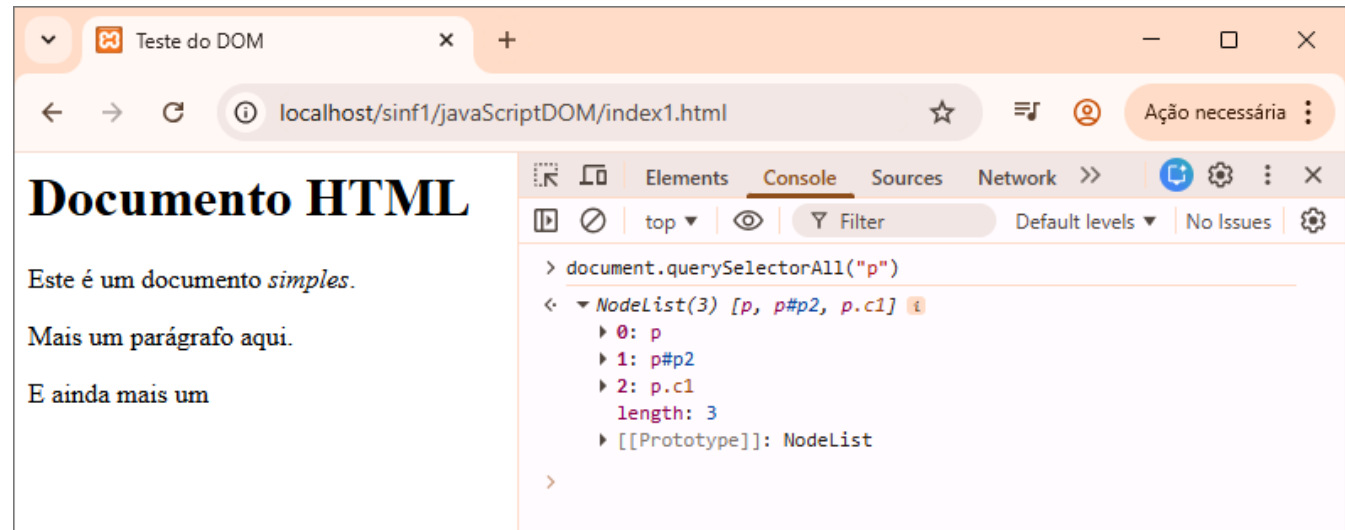


# Accessing elements: `querySelectorAll(selector)`

## `querySelectorAll(selector)`

Finds the reference to a set of HTML elements from a CSS selector

```
document.querySelectorAll("p")
```



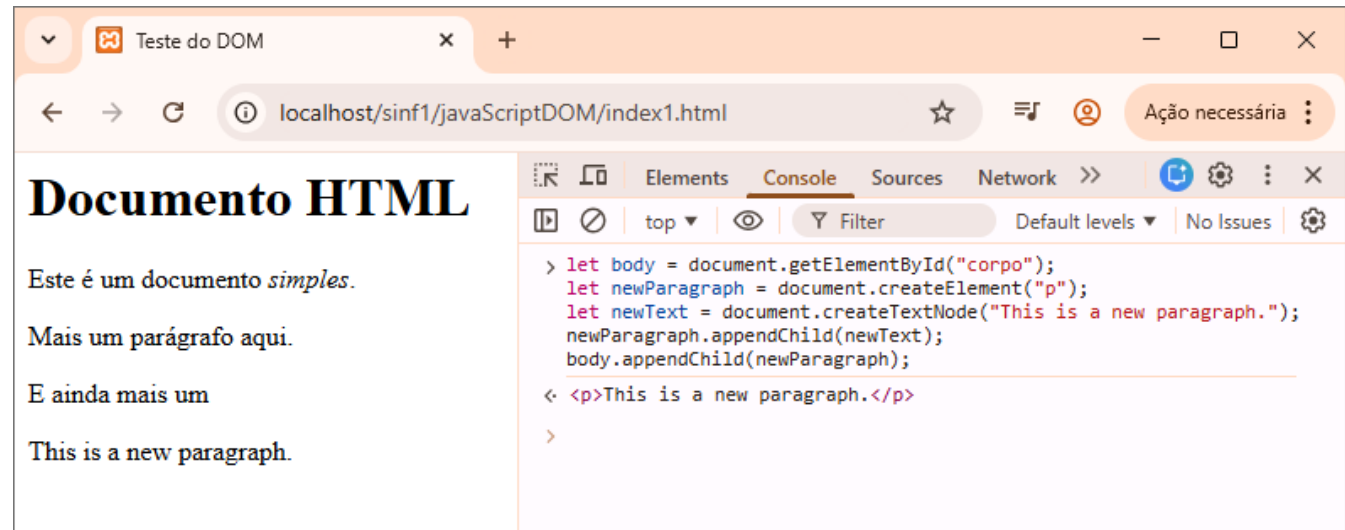
# Creating/modifying elements

**createElement(element):** creates and HTML element

**createTextNode(text):** creates a text node

**appendChild(element):** adds an HTML element

```
let body = document.getElementById("corpo");
let newParagraph = document.createElement("p");
let newText = document.createTextNode("This is
a new paragraph.");
newParagraph.appendChild(newText);
body.appendChild(newParagraph);
```

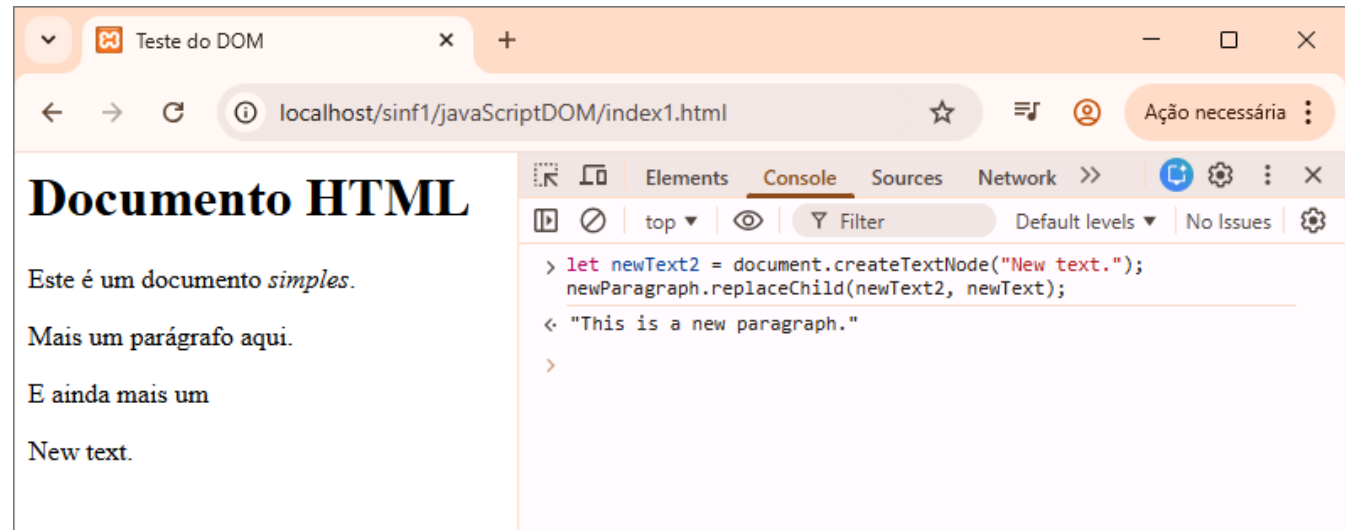


# Creating/modifying elements - replace

## **replaceChild(new, old)**

Replaces the old HTML by the new one

```
let newText2 = document.createTextNode("New text.");  
newParagraph.replaceChild(newText2, newText);
```

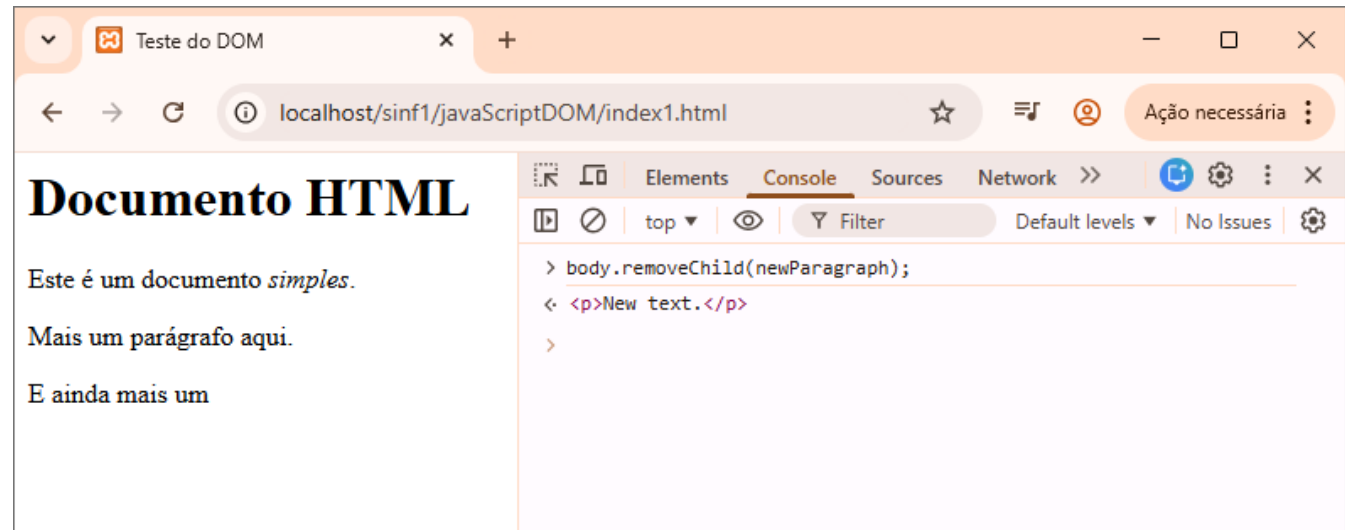


# Creating/modifying elements - remove

## **removeChild(element)**

Removes the HTML element

```
body.removeChild(newParagraph);
```



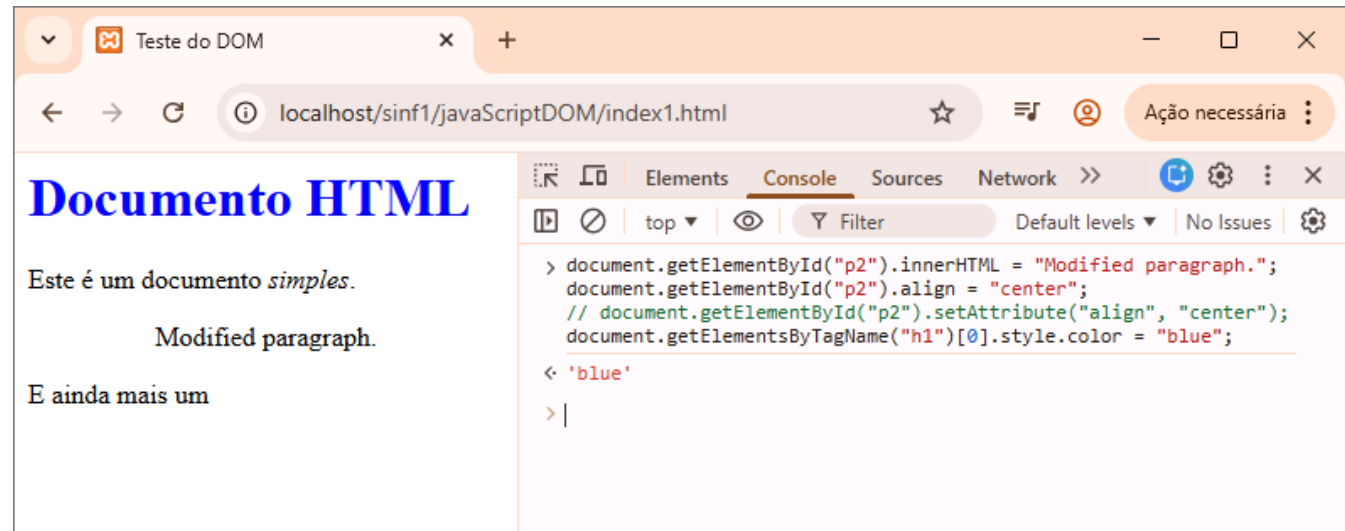
# Modifying HTML elements

...**innerHTML**: alters the HTML element's content

...**<attributeName>** ou **setAttribute(attribute,value)**: alters the HTML element's content

...**style.property**: alters the element's css in the property

```
document.getElementById("p2").innerHTML = "Modified paragraph.";
document.getElementById("p2").align = "center";
// document.getElementById("p2").setAttribute("align", "center");
document.getElementsByTagName("h1")[0].style.color = "blue";
```



# Event types

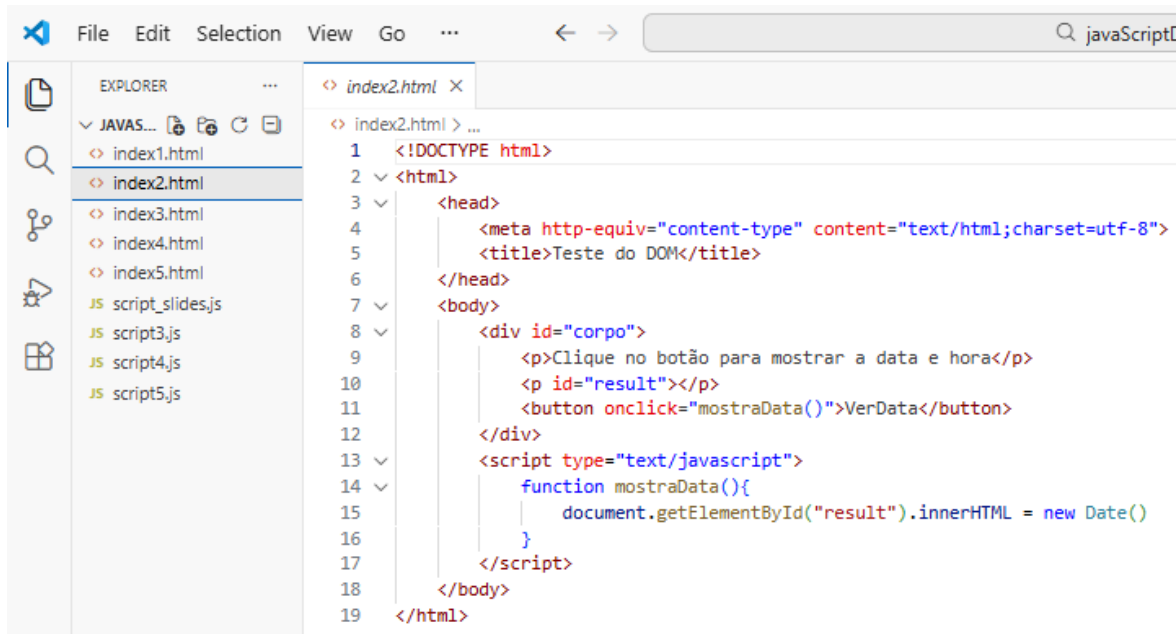
| Category | Type        | Description                |
|----------|-------------|----------------------------|
| Window   | load        | Fully available window     |
|          | unload      | End of “window available”  |
|          | resize      | Resizing the window        |
|          | scroll      | Using scroll on the page   |
| Mouse    | click       | Click                      |
|          | doubleclick | Double click               |
|          | mousedown   | Pressing the mouse button  |
|          | mouseup     | Releasing the mouse button |
|          | mousemove   | Moving the mouse           |



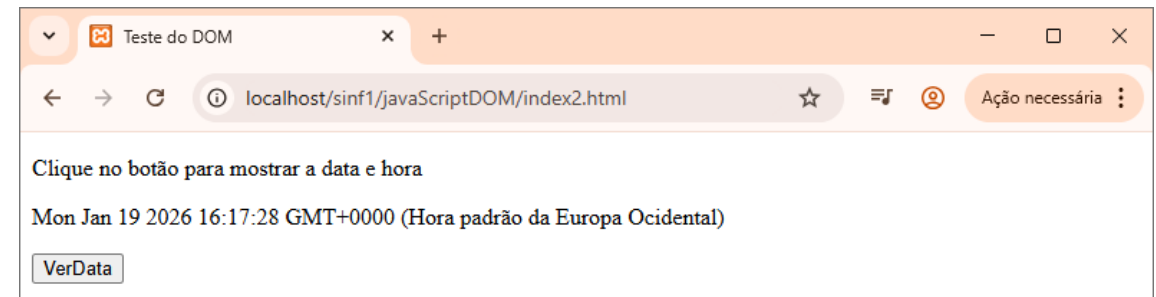
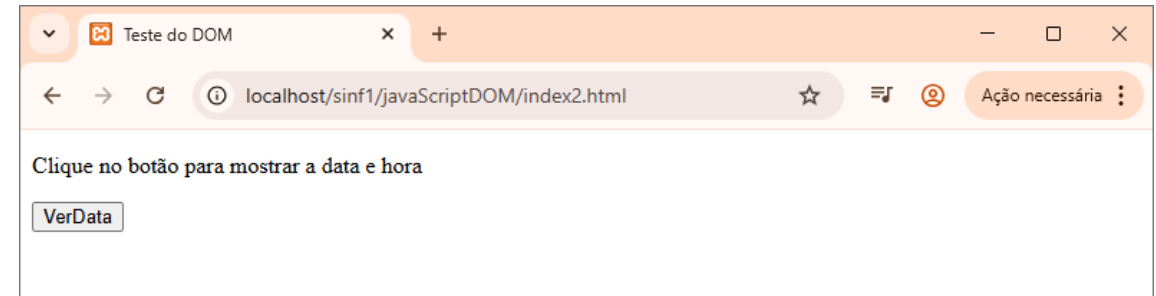
## Event types (cont.)

| Category | Type     | Description         |
|----------|----------|---------------------|
| Keyboard | keypress | Pressing key (full) |
|          | keydown  | Pressing key        |
|          | keyup    | Releasing key       |
| Elements | change   | Value change        |
|          | focus    | Receive focus       |
|          | select   | Text selection      |
|          | submit   | Form submission     |

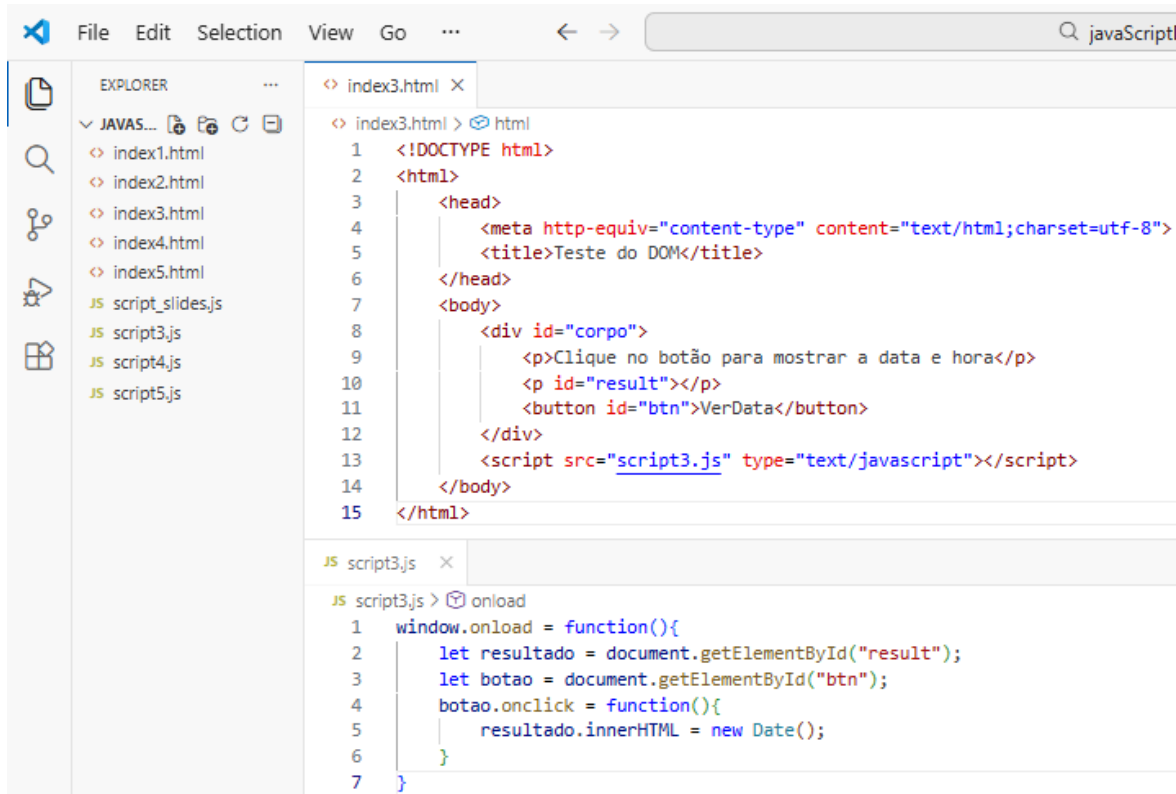
# Events: example onclick 1



```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta http-equiv="content-type" content="text/html; charset=utf-8">
5     <title>Teste do DOM</title>
6   </head>
7   <body>
8     <div id="corpo">
9       <p>Clique no botão para mostrar a data e hora</p>
10      <p id="result"></p>
11      <button onclick="mostraData()">VerData</button>
12    </div>
13    <script type="text/javascript">
14      function mostraData(){
15        document.getElementById("result").innerHTML = new Date()
16      }
17    </script>
18  </body>
19 </html>
```



# Events: example onclick 2



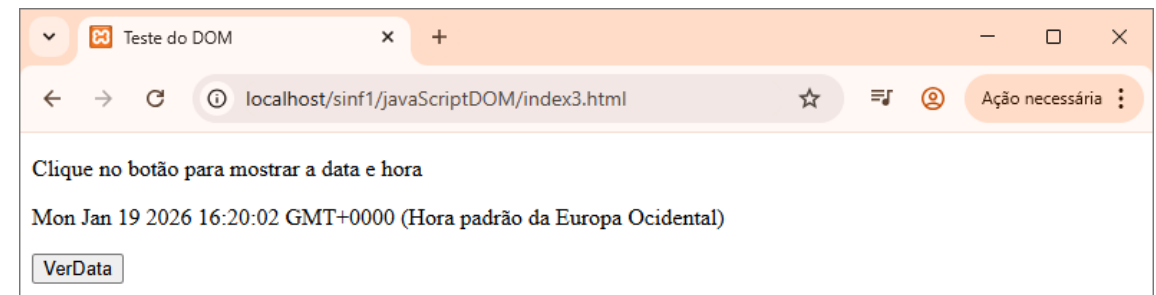
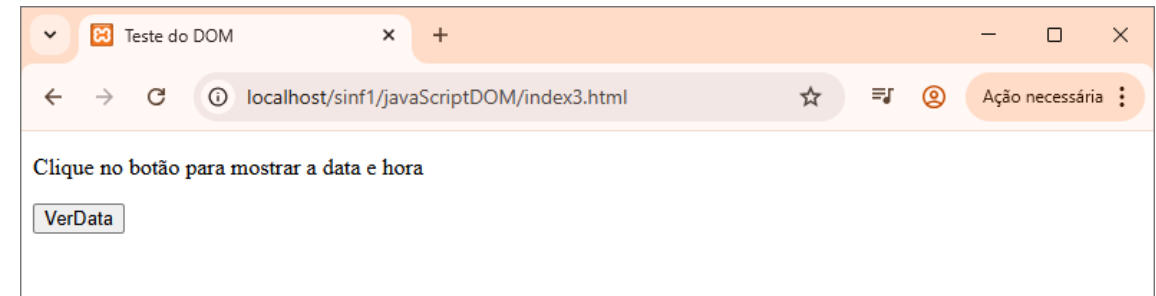
The screenshot shows the VS Code editor with two files open: `index3.html` and `script3.js`.

**index3.html** content:

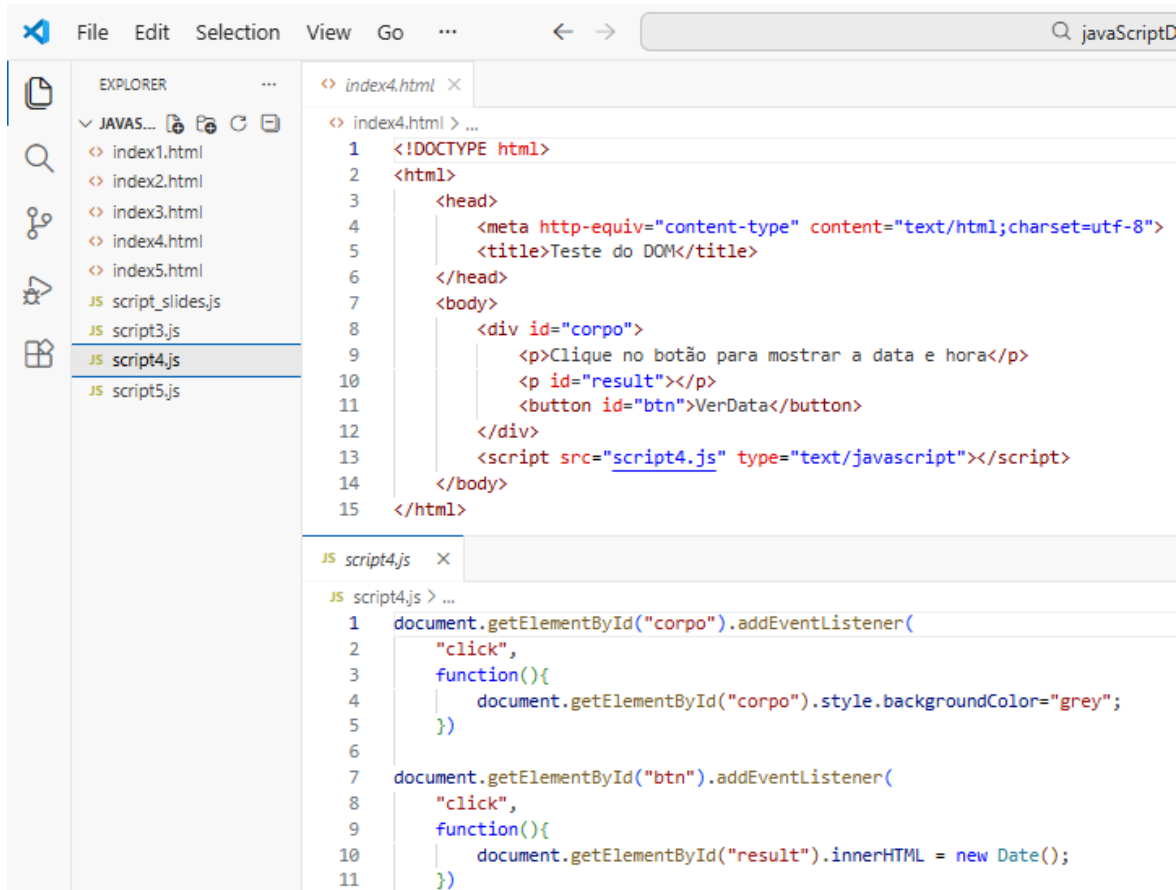
```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta http-equiv="content-type" content="text/html; charset=utf-8">
5     <title>Teste do DOM</title>
6   </head>
7   <body>
8     <div id="corpo">
9       <p>Clique no botão para mostrar a data e hora</p>
10      <p id="result"></p>
11      <button id="btn">VerData</button>
12    </div>
13    <script src="script3.js" type="text/javascript"></script>
14  </body>
15 </html>
```

**script3.js** content:

```
1 window.onload = function(){
2   let resultado = document.getElementById("result");
3   let botao = document.getElementById("btn");
4   botao.onclick = function(){
5     resultado.innerHTML = new Date();
6   }
7 }
```



# Events: example onclick 3



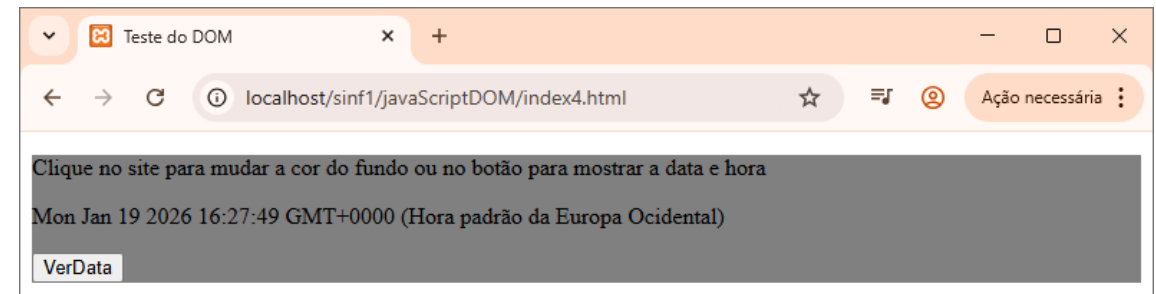
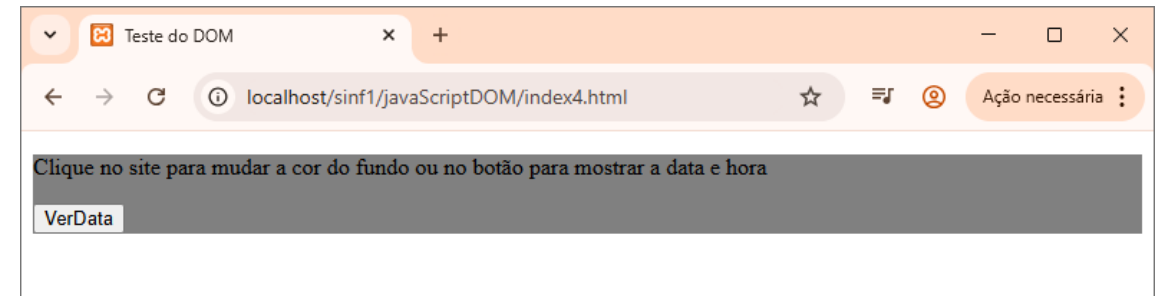
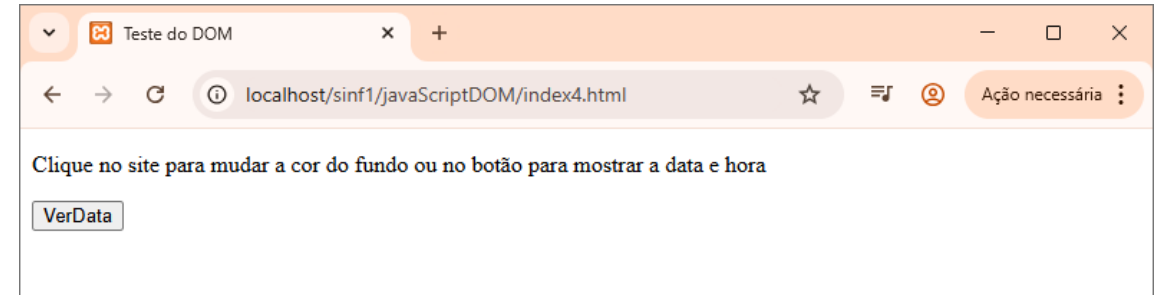
The screenshot shows the Visual Studio Code editor with two files open: `index4.html` and `script4.js`.

**index4.html** code:

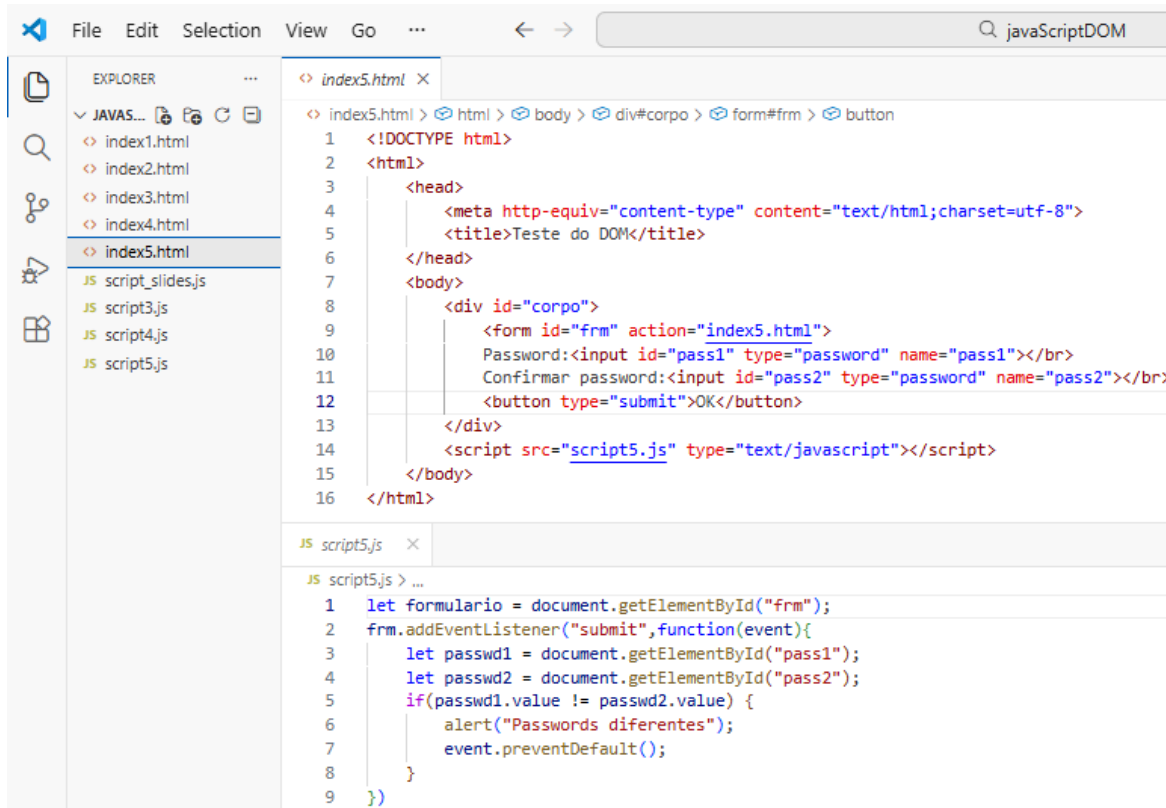
```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta http-equiv="content-type" content="text/html; charset=utf-8">
5     <title>Teste do DOM</title>
6   </head>
7   <body>
8     <div id="corpo">
9       <p>Clique no botão para mostrar a data e hora</p>
10      <p id="result"></p>
11      <button id="btn">VerData</button>
12    </div>
13    <script src="script4.js" type="text/javascript"></script>
14  </body>
15 </html>
```

**script4.js** code:

```
1 document.getElementById("corpo").addEventListener(
2   "click",
3   function(){
4     document.getElementById("corpo").style.backgroundColor="grey";
5   })
6
7 document.getElementById("btn").addEventListener(
8   "click",
9   function(){
10    document.getElementById("result").innerHTML = new Date();
11  })
```



# Events example: verify matching passwords



The screenshot shows the Visual Studio Code editor with two files open: `index5.html` and `script5.js`. The `index5.html` file contains an HTML form with two password input fields and a submit button. The `script5.js` file contains a JavaScript function that checks if the two passwords match and displays an alert if they do not.

```
<code>index5.html</code>
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta http-equiv="content-type" content="text/html; charset=utf-8">
5     <title>Teste do DOM</title>
6   </head>
7   <body>
8     <div id="corpo">
9       <form id="frm" action="index5.html">
10        Password:<input id="pass1" type="password" name="pass1"></br>
11        Confirmar password:<input id="pass2" type="password" name="pass2"></br>
12        <button type="submit">OK</button>
13      </div>
14      <script src="script5.js" type="text/javascript"></script>
15    </body>
16  </html>

<code>script5.js</code>
1 let formulario = document.getElementById("frm");
2 frm.addEventListener("submit",function(event){
3   let passwd1 = document.getElementById("pass1");
4   let passwd2 = document.getElementById("pass2");
5   if(passwd1.value != passwd2.value) {
6     alert("Passwords diferentes");
7     event.preventDefault();
8   }
9 })
```

